FinBank 프로젝트 진행 보고서

금융 도메인 특화 백엔드 포트폴리오 — 완료 내역 · 코드 구조 · 남은 로드맵 (2026-07-28 기준)

---

1. 프로젝트 전략 (변하지 않는 두 원칙)

① **깊이 우선(넓이보다)** — 기능을 여러 개 얕게 만들지 않고, 금융 SI가 가장 원하는 여신(대출) 도메인 하나를 실무 수준의 깊이로 판다.

② **기술은 도메인이 끌어올 때만 도입** — "이거 없으면 뭐가 안 되나"에 한 문장으로 답하지 못하는 기술은 넣지 않는다. (QueryDSL은 동적 검색이 생기는 C4에서, Spring Batch는 연체 정산이 생기는 D에서. Redis 분산 락은 다중 인스턴스를 증명할 수 없어 보류)

세션 운영 규칙: 각 단계 끝마다 **숫자 하나 + 트러블슈팅 하나 + README 갱신**, 그리고 언제 지원 시점이 와도 되도록 **항상 배포 가능한 상태 유지**.

---

2. 전체 로드맵과 현재 위치

단계	내용	상태
A	동시성 숫자화 — 락 없음 vs 비관적 락 벤치마크	완료
B	금액 BigDecimal 전환 (DECIMAL(19,4) + MoneyPolicy)	완료
C0	ErrorCode 기반 예외 체계 (409/422 구분)	완료
C1	여신 스키마·엔티티·자동심사·신청 API + 프론트 대출 탭	완료
—	도메인형 패키지 재배치 (account/auth/member 분리)	완료
C2	이자 계산 엔진 3종 + 연체이자·중도상환수수료 + 3층 검증	완료
C3	**대출 실행·상환 API + 멱등키 + 동시성 증명**	**완료**
—	프론트 실행·상환 연동 (멱등키 발급 포함)	완료
E-1	GitHub Actions CI (빌드+테스트 자동화) — 앞당겨 도입	완료 (PR #18에서 통과 확인)
C4	**QueryDSL 동적 검색 + 인덱스 성능 측정**	**완료**
D	**Spring Batch — 연체 감지·연체이자 가산 배치**	**완료**
E-2	**마감 — 테스트 격리, Docker Compose, README 종합**	**완료**

여신 파트(C)의 핵심인 실행·상환이 끝나면서 **돈이 도는 전 구간(신청 → 심사 → 실행 → 상환 → 완납)이 화면까지 연결**됐다. 남은 C4·D는 성능과 배치라는 다른 축이다.

---

3. 이번 단계(C3)에서 한 일

3-1. 멱등키 — "두 번 눌러도 한 번만 나간다"

사용자가 상환 버튼을 두 번 누르거나 네트워크 타임아웃으로 클라이언트가 재시도하면 같은 의도의 요청이 서버에 두 번 도착한다. 클라이언트가 요청마다 고유 키(UUID)를 보내고, 서버는 같은 키의 두 번째 요청에 **저장해둔 응답을 그대로** 돌려준다.

핵심은 트랜잭션 경계다.

단계	전파 속성	이유
키 선점(INSERT)	`REQUIRES_NEW`	본 처리와 같은 트랜잭션이면 롤백 시 키도 사라져 동시 재시도를 못 막는다
응답 확정(COMPLETED)	`MANDATORY` (본 처리와 동일 트랜잭션)	"돈이 움직인 사실"과 "응답 스냅샷"이 같은 커밋에 묶여야 한다

단계	전파 속성	이유
실패 시 해제(DELETE)	`REQUIRES_NEW`	롤백되는 트랜잭션에 얹으면 삭제도 함께 사라진다

선점의 승패는 `idempotency_keys.idem_key` **UNIQUE 제약**이 결정한다. "조회 후 없으면 INSERT"는 동시 요청 둘 다 통과하지만 UNIQUE는 반드시 하나만 통과시킨다 — 경쟁의 중재를 애플리케이션이 아니라 DB에 맡기는 이유다. 같은 키로 다른 내용을 보내면 요청 해시(SHA-256)가 달라 422로 거부한다.

3-2. 이중 처리 방어 — 층마다 막는 것이 다르다

사고 유형	1차 방어	최종 방어
같은 신청 이중 실행	조건부 상태 전이 `UPDATE ... WHERE status='APPROVED'` (0행이면 409)	`loan_accounts.application_id` UNIQUE
같은 회차 이중 상환	조건부 전이 `UPDATE ... WHERE status IN ('PENDING','OVERDUE')`	`loan_transactions.idempotency_key` UNIQUE
버튼 두 번 / 재시도	멱등키 선점	위 두 제약

조건부 UPDATE가 핵심인 이유: MySQL 기본 격리수준(REPEATABLE READ)에서 일반 SELECT는 트랜잭션 시작 시점의 스냅샷을 읽는다. 락을 기다렸다 통과한 두 번째 요청이 "이미 상환된 회차"를 아직 PENDING인 것으로 볼 수 있다는 뜻이다. 반면 UPDATE는 항상 최신 커밋본을 기준으로 조건을 평가하므로 영향 행 수 0으로 정확히 막힌다. **상태 검사와 상태 변경을 DB의 한 문장으로 합치는 것**이 check-then-act 경쟁을 없애는 방법이다.

3-3. 데드락 방지 — 자원 유형 간 락 순서 고정

상환은 계좌와 대출 두 자원을 동시에 잠근다. 전역 규칙은 **`Account` → `LoanAccount` 순서**(이체의 "ID 오름차순" 규칙과 같은 목적). 그런데 연결계좌 ID는 대출을 읽어야 알 수 있으므로, 대출을 **엔티티가 아니라 ID만 프로젝션으로** 먼저 읽는다. 엔티티를 먼저 읽으면 1차 캐시 때문에 뒤이은 락 조회가 락 이전의 낡은 값을 돌려준다 — Phase A에서 확인한 함정의 재적용이다.

3-4. 동시성 증명 (이번 단계의 숫자)

조건: 원금 1,000만 / 연 5.9% / 12개월 원리금균등 → 1회차 총액 **860,205원**. 100개 스레드가 **같은 회차**를 동시에 상환 시도.

시나리오	성공(응답)	409 충돌	기타 예외	상환 원장	계좌 출금	잔액 감소
서로 다른 100개 요청	**1**	99	0	1건	1회	860,205원
같은 멱등키 100개 재시도	70 (1회 처리 + 69회 응답 재생)	30	0	**1건**	**1회**	860,205원

두 번째 줄이 멱등키의 존재 이유다. 100개 요청 중 **70개가 성공 응답을 받았지만 돈은 한 번만 움직였다** — 69개는 저장된 응답의 재생이고, 30개는 선점 경쟁 중(IN_PROGRESS)이라 409를 받았다. 락 대기 타임아웃이나 데드락 같은 "기타 예외"는 0건이며, 계좌 잔액 감소액은 회차 총액과 1원도 다르지 않다.

3-5. 중도상환

전액 중도상환은 **잔여원금 + 경과이자 + 중도상환수수료**를 한 번에 출금한다.

- 경과이자 = 잔여원금 × 약정금리 × (직전 납부 기준일 ~ 상환일) / 365
- 수수료 = 잔여원금 × 수수료율 × 잔존일수 / 총 대출일수 (잔존기간 체감 — 만기일 상환 시 0원)
- 수수료율도 **실행 시점 스냅샷**이다. 은행이 상품 수수료율을 바꿔도 기존 계약 조건은 불변이어야 한다.
- 남은 회차는 삭제하지 않고 `CANCELED`로 보존한다 — "계약이 원래 어떤 계획이었는지"는 사후 검증에 필요한 사실이다(원장 append-only 철학의 연장).
- 연체(OVERDUE) 대출은 중도상환을 명시적으로 거부한다. 연체이자 정산이 D단계 배치의 몫이기 때문이다.

3-6. 프론트 연동 — 멱등키를 화면이 어떻게 다뤄야 하는가

기존 스텁은 API 호출마다 `crypto.randomUUID()`로 새 키를 만들고 있었다. 그러면 재시도할 때마다 서버 입장에서는 "처음 보는 요청"이라 **멱등이 전혀 걸리지 않는다**. 올바른 방식은 **사용자 행동 1회 = 키 1개**다. 버튼을 누른 순간 키를 만들고, 네트워크 실패로 재시도할 때는 같은 키를 그대로 다시 보낸다.

- `ApiError`에 HTTP 상태와 서버 에러 코드를 실어 409(이미 상환됨 / 처리 중)와 422(키 재사용)를 화면 문구로 구분
- 신청 이력 카드에 실행 버튼(APPROVED일 때만), 하단에 상환 패널 — 계약 요약 + 전 회차 스케줄 표 + 회차 상환 / 전액 중도상환
- 실행·상환 후 대출·스케줄·신청 이력·**계좌 잔액**까지 동기화

3-7. CI (E단계 앞당김)

테스트가 313개까지 커졌고 앞으로 C4·D가 더 붙는다. 수동 실행에 의존하면 언젠가 깨진 상태로 쌓이므로 GitHub Actions를 먼저 도입했다.

- push(main/develop)·PR마다 백엔드 빌드+테스트, 프론트 빌드
- 통합 테스트는 **실제 MySQL 컨테이너**에서 실행 — 비관적 락, 조건부 UPDATE, REPEATABLE READ 스냅샷은 대부분 DB 엔진의 특성이라 H2로 바꾸면 검증 자체가 무의미해진다
- 잡마다 컨테이너가 새로 뜨므로 로컬 DB 상태에 오염되지 않는다 (Flyway 유령 마이그레이션 사고의 구조적 해법)

3-8. C4 — 동적 검색과 인덱스 (이번 단계의 숫자)

조건 7개(상태·상환방식·상품·실행일 구간·원금 구간)를 자유 조합하는 검색을 QueryDSL로 구현했다. 값이 없는 조건은 `null`을 반환해 `where()`가 건너뛰므로, 조합이 100가지를 넘어도 코드에는 조건 하나당 메서드 하나만 남는다.

측정 조건: 한 회원이 대출 10만 건 보유 / "최근 30일 안에 실행된 정상 대출"(전체의 약 2.7%)

쿼리	인덱스	사용 인덱스 (type)	조사 행 수	평균 응답
COUNT	미사용	(없음) ALL — 풀스캔	99,400	26.83ms
COUNT	복합 인덱스	idx_…_member_status_executed (range)	**2,664**	**1.13ms**
목록+정렬	미사용	PRIMARY (index)	10	0.54ms
목록+정렬	복합 인덱스	PRIMARY (index)	373	0.54ms

조사 행 수 37배 감소, 응답시간 24배 단축.

컬럼 순서 (`member_id, status, executed_at`)의 근거는 "범위 조건 뒤의 컬럼은 탐색 범위를 좁히지 못한다"이다. 등치 조건을 앞에, 범위 조건을 마지막에 둔다.

정렬이 붙은 목록 쿼리는 양쪽 모두 PK를 탔다 — `ORDER BY id DESC LIMIT 10`이면 옵티마이저가 "PK 역순으로 훑다가 10건 채우면 끝"을 택하기 때문이다. 지금은 조건 일치율이 2.7%라 빠르지만 데이터 분포에 의존하는 계획이므로, 커버링 인덱스나 커서 페이징이 필요한 지점으로 기록해 뒀다.

3-9. D — 연체 정산 배치 (이번 단계의 숫자)

매일 밤 **연체 감지 → 연체이자 가산** 순서로 두 배치가 돈다.

설계의 핵심은 Reader다. 이 배치들은 읽은 행의 상태를 바꿔 자기 조회 조건에서 스스로 제외시키므로, offset 페이징을 쓰면 앞 행이 빠진 만큼 뒤가 당겨져 **통째로 건너뛴다** — 그러고도 배치는 정상 종료한다. "마지막으로 읽은 id 다음부터" 읽는 커서 방식으로 해결하고 청크 경계를 넘는 250건 테스트로 고정했다.

멱등은 세 겹이다: ① Spring Batch가 같은 파라미터(기준일)의 재실행을 거부 ② 감지는 `status = PENDING` 조회 조건 ③ 가산은 `lastAccruedAt < 기준일` 조회 조건. 원리는 같다 — 처리 완료 표시를 데이터에 남기고 그 표시를 읽기 조건에 넣는다.

청크 사이즈별 처리시간 (5만 건 연체 감지)

청크 크기	처리시간	커밋 횟수	건당
100	37,969ms	500	759µs
500	12,418ms	100	248µs
1000	**9,927ms**	50	198µs

100 → 1000으로 **3.8배** 빨라졌지만, 100→500이 3.1배인 반면 500→1000은 1.25배로 수확이 급격히 체감한다. 커밋 비용이 지배적인 구간을 벗어나면 그다음은 쿼리·영속성 컨텍스트 비용이기 때문이다. 청크를 키울수록 실패 시 롤백 범위와 메모리 사용이 늘어나므로 정답이 아니라 트레이드오프이며, 기본값 100을 유지하되 잡 파라미터로 열어뒀다.

---

4. 코드 구조

```
com.finbank.backend
 ├── account/    controller · service(입출금·이체) · domain(Account, TransactionLog) · repository · dto
 ├── auth/      controller · service · security(JWT 필터+토큰) · domain(RefreshToken) · repository · dto
 ├── member/      domain(Member) · repository · service(CurrentMemberProvider)
 ├── loan/        controller(신청/실행·상환) · service(신청·심사 / 실행 / 상환 / 조회)
 │                · domain(+calculator 이자·연체·수수료·경과이자 엔진) · repository · dto · config
 ├── common/      MoneyPolicy(반올림 정책), PageResponse, idempotency(멱등키 저장소·실행기)
 ├── config/      SecurityConfig, SwaggerConfig
 └── exception/   ErrorCode, ApiError, BusinessException, GlobalExceptionHandler
```

실행·상환 서비스는 각각 **두 개의 빈**으로 나뉜다(LoanExecutionService + LoanExecutionProcessor). 멱등키 선점/해제는 본 처리와 다른 트랜잭션이어야 하는데, 한 클래스 안에서 자기 메서드를 호출하면 프록시를 타지 않아 트랜잭션 경계가 적용되지 않기 때문이다. "경계가 다른 두 책임"이라 두 빈으로 분리했다.

---

5. 트러블슈팅 기록 (문제 → 원인 → 해결)

#	문제	원인	해결
1	BigDecimal 전환 후 기존 테스트 전멸	DB DECIMAL(19,4)는 1000.0000으로 조회되는데 equals는 스케일까지 비교	비교를 전부 compareTo 계열로 (테스트는 isEqualByComparingTo)
2	Flyway 체크섬 불일치 + 새 V3가 실행 안 됨	로컬 DB에 과거 실험 브랜치의 V3·V4 기록 잔존(유령 마이그레이션)	로컬 한정 clean 후 재마이그레이션. 교훈: 적용된 마이그레이션 파일은 수정 금지, 공유 DB면 forward-fix
3	여신 테이블 추가 후 기존 테스트 7개 FK 위반	각 테스트에 흩어진 DB 정리 로직이 새 FK를 모름	삭제 순서를 DatabaseCleaner 한 곳으로 일원화
4	외부 계산기와 회차 이자 최대 9원 차이	계산기는 이자 10원 미만 절사, 우리는 원 미만 절사	정책 차이로 규명(월 상환금 완전 일치가 증거) — README에 분석 기록
5	**코드를 안 고쳤는데 어느 날 테스트가 깨짐**	테스트 계좌번호가 "bench-" + System.nanoTime() — 머신 가동시간에 따라 자릿수가 늘어 VARCHAR(20) 초과	계좌번호 형식을 TestAccountNumbers 한 곳으로 모아 프로덕션과 같은 12자리로 고정

#	문제	원인	해결
8	테스트 설정 파일을 추가하자 통합 테스트 53개 전부 접속 실패	main·test 모두 `classpath:/application.yml` — 같은 이름이라 병합이 아니라 **대체**된다. URL 한 줄만 남고 계정·JPA·Flyway 설정이 사라짐	설정 파일 대신 Gradle 테스트 태스크의 시스템 프로퍼티로 URL만 주입 (우선순위는 높고 나머지는 보존)
7	**다 갚아도 연체이자가 계속 붙는 구조**	상환이 회차·대출 상태만 되돌리고 `OverdueHistory`를 닫지 않음. C3에는 이력을 만드는 주체가 없어 증상이 없었다	상환 시 이력 `resolve` + "상환 → 가산 중단" 통합 테스트로 고정
6	성능 측정용 인덱스 DROP이 거부(에러 1553)	V6로 복합 인덱스를 만들자 MySQL이 FK 전용 자동 인덱스(`fk_loan_acc_member`)를 제거 → 복합 인덱스가 FK를 지탱하는 유일한 인덱스가 됨	스키마 조작을 버리고 `IGNORE INDEX` 힌트로 전환. 재려던 것은 "인덱스가 없을 때"가 아니라 "옵티마이저가 못 쓸 때"였다

5번은 "가끔 깨지는 테스트"로 취급되면 신뢰를 잃는 유형이다. 테스트 픽스처도 프로덕션과 같은 길이·형식 제약을 따라야 한다는 교훈으로 남겼다.

---

6. 테스트 현황

영역	종류	개수
계좌·이체	동시성(CountDownLatch)·데드락·벤치마크·원장·이체 룰	16
예외 체계	ErrorCode 매핑 단위 테스트	3
여신 신청·심사	상태 전이 단위 6 + 심사 룰 통합 8	14
이자 엔진	확정 기대값 13 + 속성 240 + 연체·수수료 8	261
대출 실행 (C3)	정합성·멱등·이중 실행·권한·키 해제	**7**
상환·중도상환 (C3)	순서 규칙·멱등·완납·잔액 부족 롤백·수수료	**9**
상환 동시성 (C3)	100스레드 × 2 시나리오	**2**
경과이자 계산기 (C3)	확정 기대값 단위 테스트	**5**
대출 검색 (C4)	단일 조건·조건 조합·경계 포함·타인 대출 미노출·페이징	**8**
연체 감지 배치 (D)	감지 정확성·멱등·청크 경계 누락 없음·기준일 파라미터	**5**
연체이자 가산 배치 (D)	계산 일치·멱등·구간 연속성·밀린 배치 따라잡기·상환 연동	**7**
합계		**333**
성능 측정 (C4·D)	10만 건 인덱스 전후 / 5만 건 청크 사이즈 비교 — `perf` 태그, CI 제외	2

원칙: 개수보다 위험도. 돈이 움직이는 경로(동시성·이자 계산·멱등)에 테스트를 집중한다. 전 항목 통과(로컬 MySQL 8.0 / JDK 17).

---

7. 앞으로 할 일

C4 — QueryDSL 동적 검색 + 인덱스 (다음 차례)

대출 목록을 상태/기간/금액/상품 조건 조합으로 검색하는 화면이 생기면서 QueryDSL 도입 근거가 선다(문자열 JPQL 동적 조립 대신 컴파일 타임 타입 안전). 시드 10만 건을 넣고 인덱스 전/후 실행계획(EXPLAIN)과 응답시간을 측정해 숫자를 남긴다.

연체 배치용 인덱스(`status+due_date`)는 이때 미리 깐다.

D — Spring Batch (연체 정산)

매일 밤 도는 배치 2개: ① 연체 감지 — 납기 지난 PENDING 회차를 OVERDUE로 전이하고 연체이력 생성 ② 연체이자 가산 — C2 계산기로 일할 이자를 누적. `last_accrued_at` 컬럼으로 배치를 재실행해도 이중 가산이 안 되게 설계(배치 멱등 — C1에서 컬럼을 미리 깔아둠). 청크 사이즈 100/500/1000 처리시간 비교로 숫자 확보, 대량 처리 시 OOM을 피하는 페이징 Reader 스토리 포함.

E — 마감

통합 테스트 격리(별도 스키마), Docker Compose(App+MySQL), README 종합 정리(기획→ERD→이자 수식+검증→동시성/배치 비교표→트러블슈팅→Swagger). 선택 확장: 이체 수신 알림의 SSE 전환(폴링→푸시, 도입 근거가 명확한 자리).

---

8. 운영 방식 요약

브랜치는 “독립적으로 머지 가능한 단위”로 나눈다(기간 아님) — 여신 1단계는 뼈대+신청까지 한 PR, 이자 엔진은 별도 PR, 파일 이동은 반드시 단독 PR. 커밋 메시지는 한국어(feat/fix/refactor/test/docs/ci 접두어), 브랜치명은 영어. 매 단계 종료 시 테스트 전체 통과 + README 갱신 + 배포 가능 상태를 확인한다. 이제 이 확인의 절반은 CI가 자동으로 한다.